HANDOFF — ABRA, 2026-07-28 (overnight session)

Read this **after** `HANDOFF-2026-07-27.md`, which it corrects in several places. Repo: `C:\Users\willj\Projects\Pokemon\ABRA` Suite: `SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown node tests/run-all.js` → **19 passed, 3 failed, 1 skipped.** The 3 failures are the same deliberate ones as before.

---

0. THE PREVIOUS HANDOFF WAS WRONG ABOUT THREE THINGS

Recorded first, because they were acted on as fact.

Armor Tail / Queenly Majesty / Dazzling were NOT absent. They were already modelled at 99% and 98% through the measured `ability-blocks.json` table, which `abilityBlockProb` applies. The handoff said "grep returns zero", and it does — `board.js` deliberately names almost nothing. **Grep is not a coverage test for this file.** Only the *ally-side* half was genuinely missing, and that is now fixed.

Contrary does NOT invert `myOffenseStage`. That feature reads stat stages already on the board, recorded from real protocol events, and is correct whatever produced them. Contrary changes the *predicted* effect of a move under consideration — `movesBoostMe` and `movesLowerFoe`, and only those.

`pory.py` **is not one of the raw-store readers.** It filters to clean ids via `store.load_games(clean=True)` and refuses to run without them. `pory_baseline.py` and `train_value.py` are on that list; PORY itself is not.

---

1. WHAT LANDED

The corpus grew 35%, and every earlier number is stale

The forfeit rule now excludes only forfeits **before any action**, on Will's ruling. The old rationale ("a forfeit records who quit, not who was winning") was never tested and is false: of 1,528 open-sheet forfeits with at least one action, the player who quit was **behind on mons 86.8%** of the time and **ahead 0.9%**. It was discarding 1,528 decided games to exclude 4 undecided ones.

clean open-sheet 2,114 -> 2,860 clean ladder 2,653 -> 3,571

Mean game length moves 8.46 → 8.09 turns; median (8) and p90 (12) do not move. **Anything computed on a clean corpus before 2026-07-28 was computed on a different population.**

The stores are tracked compressed

`games.ladder.jsonl` was 84.6 MB against GitHub's **hard** 100 MB limit — about 38 hours of collection from the point where every push fails, including the ingest Action's. All three stores now track as `.gz` (137.6 MB → 15.4 MB). `quality.js` / `quality.py` read either form, preferring the plain file when present. `build/compress-stores.js --check` reports staleness.

board.js batch — one refit, seven changes

- **Speed is investment-aware.** It was base stats; `spreadLines()` already existed and the *damage* calc already used it. The error is large and non-uniform: Garchomp 102→167, Whimsicott 116→179, but Kingambit 50→73. Base stats compress the range and flip orders.
- **Choice Scarf, paralysis, and the four weather-speed abilities** now reach move order, all probed from `onModifySpe` rather than listed.
- **Side-wide blockers protect their partner.** Classified by handler name (`onFoeTryMove`).
- **Megas are resolved from the sheet's stone.** 27.5% of sheet entries hold one; their abilities were being read off the base forme.
- **Contrary/Simple** via `onChangeBoost`.
- `stallIntoEncore` — Will's feature, and it is real (see §2).

PORYGON2 — a new value function, deliberately not a new PORY

k-NN over self-play positions, evaluated on held-out **human** games. **62.8%** against **60.5%** for the material baseline, and better on Brier. PORY is untouched so the two stay separately quotable.

The risk lever (`engine/variance.js`)

Seek variance as an underdog, decline it as a favourite. Credit: Nate Silver, *On the Edge*. Defaults to an exact no-op and must be switched on with a stated belief.

---

2. WHAT IS VERIFIED, AND WHAT IS NOT

VERIFIED

- `stallIntoEncore` **−1.171** [−1.686, −0.655] — 12th of 48 weights, largest mover under reweighting (−1.993). Will's domain claim is a measured regularity.
- Ally-side blocking: Fake Out at a Sinistcha reads 0.000 beside Garchomp, **0.992** beside Farigiraf, 0.984 beside Tsareena.
- Contrary: Swords Dance on Staraptor reads 1.000 bare, **0.000** holding Staraptite.
- Held-out fit: 30.4% top-1, −1.7694 logL, against 22.8% for the behaviour clone.
- Simulator speed: **60 ms** per complete game with a random policy, **318 ms** with MAG.

MEASURED NEGATIVES — these are results, not gaps

- **PORYGON2 plateaus.** 8x the training data moved accuracy **−0.2 points**. Saturation at ~9,000 positions. More self-play is *not* the lever; the ceiling is the feature set.
- **Weighting the k-NN distance by learned importance made it worse** (62.5% vs 62.8%). It amplifies material and crushes the features the k-NN was using non-linearly.
- **Fixing four real mechanics barely moved opponent prediction** — joint recall at k=5 went 32.8% → 33.3%. The opponent-prediction ceiling is not missing mechanics.
- **MAG cannot narrow the opponent's turn.** Joint recall 33.3% at k=5, 74.5% at k=20 against ~56 joint actions. There is no k that is both affordable and honest.

NOT VERIFIED — do not quote

- Whether the risk lever wins games. The falsifier is the experiment: assert a gap you do not have against an equal opponent; it should **lose**. If it wins, the model is wrong.

- MAG against a human. Still never measured (task 24).
- Everything in `HANDOFF-2026-07-27.md` §4 marked NOT VERIFIED.

---

3. TRAPS — THE OLD ONES STILL APPLY, PLUS THREE

Never edit `board.js` while a fit or self-play run is in flight. Unchanged and still true.

Grep is not a coverage test. `board.js` names almost nothing on purpose. Use `docs/MECHANICS-COVERAGE.md` (generated by `engine/mechanics_coverage.js`), which reports per *channel*. 192 abilities in this format: 92 reachable (71.5% of usage), **98 with dex handlers and no channel (22.4%)**.

An ignore rule does not apply to a file git already tracks. `data/games.ots.jsonl` was in `.gitignore` *and* tracked, because it was committed before the rule was written. The rule silently did nothing for 30.4 MB. `git rm --cached` is what makes it take effect.

Duplicate games are corrosive to a k-NN specifically. `mew.js` enumerates matchups deterministically from its seed, so two runs to the same `--out` produce the same games twice. For a logistic that is a doubled sample; for a nearest-neighbour model an exact duplicate is its own neighbour at distance zero carrying its own outcome, so the model *appears* to predict well by retrieving copies of the answer. `porygon2.py` now dedupes by id.

---

4. THE COLLINEARITY, WHICH IS NOW THE BIGGEST KNOWN DEFECT

`koTarget` is **−0.206** in the fit. Read literally that says people avoid killing. Measured model-free on held-out decisions, humans pick a killing move **1.41x** the base rate, a move that kills and moves first **1.82x**.

Will's diagnostic — fit each feature alone — settles it:

koTarget +0.764 alone -> -0.206 in fit SIGN FLIPS koFirst +0.902 -> +0.089 absorbed killsThreat +0.584 -> -0.098 SIGN FLIPS dmgFrac +0.648 -> -0.004 SIGN FLIPS movesFirst +0.412 -> +0.127 absorbed <- NEW this session

The family is inter-correlated at 0.5–0.67 and `lambda` selected on held-out data is **0**, so the coefficients are unconstrained. `eff4` and `immune`, which correlate with nothing, do not move at all.

`movesFirst` joining the block is the instructive part: it was *improved* this session, and improving it is what pulled it in. **Feature quality does not rescue a collinear design.**

The model predicts fine. The individual weights inside that block are not statements about the game and must not be quoted. Fix is ridge, or collapsing the block — neither attempted.

---

5. SLOWKING'S INTERVALS ARE UNUSABLE — DIAGNOSED, NOT FIXED

The intervals do not contain their own point estimates:

exploitability nash 0.0003 CI [0.0006, 0.0033] greedy - nash gap 0.3887 CI [0.0244, 0.3777]

Cause: `slowking_preview.py:145` **solves with** `iters=15000`**; line 157 solves every bootstrap replicate with** `iters=1000`**.** Regret matching converges with iterations, so the replicates are under-converged and their residual exploitability is solver error, not matrix uncertainty.

Not fixed because `tests/test-slowking.py` asserts on these numbers and the published non-transitive-cycle claim rests on them. **That is Will's call.** Until then treat both intervals as unusable.

---

6. WHAT I WOULD DO NEXT, IN ORDER

1. **Read the falsifier result** (`engine/mew.js --risk-a`). It decides whether `engine/variance.js` stays or goes.
2. **Fix the collinearity** — ridge, or collapse the kill block to one feature. It is the largest known defect and it now affects five features.
3. **Give PORYGON2 the conversion terms** — `koTarget` , `movesFirst` , type matchup. The learning curve says data is not the lever and the feature set is. This is the single highest-value item.
4. **Decide the SLOWKING CI** (§5).
5. **Measure MAG against a human** (task 24). Still the only thing that would tell you whether any of this plays.

Do not spend compute generating more self-play for PORYGON2. That was the plan and it is measured wrong.